7. Sözde Koddan (Pseudocode) Gerçek Koda: AI ile Algoritma Tasarlamak

Klavye Başına Geçmeden Önce Düşünceyi Yapılandırma Rehberi
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zekaya doğrudan kod yazdırmak yerine neden önce adım adım sözde kod (pseudocode) tartıştırmalıyız?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Sözde kod (Pseudocode), insan zihni ile yapay zekanın akıl yürütme motoru arasındaki ortak dildir. - 📌 Doğrudan kod istemek modelin aceleci varsayımlar yapmasına; sözde kod ise mantık adımlarını önceden planlamasına yol açar. - 📌 İyi bir sözde kod dile bağımsızdır; Python, Go veya Rust'a eşit kolaylıkla derlenebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir inşaat şantiyesine gittiğinizi düşünün. Şef mühendis ustaların yanına gelip: *"Hadi hemen tuğlaları üst üste koymaya başlayın, bina yükselsin!"* der mi? Tabii ki hayır! Önce mavi ozalit planlar (blueprints) incelenir, kolonların taşıma kapasitesi hesaplanır.

Kodlama dünyasında sözde kod (pseudocode), o mavi ozalit plandır.

Junior yazılımcıların en büyük hatası, problemi tam anlamadan yapay zekaya *"Hemen bana kod yaz"* demektir. Oysa önce mantık akışını sözde kodla kurduğunuzda, yapay zeka hem mantık hatalarını hemen fark eder hem de yazdığı kod ilk seferde çalışır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Sözde Kod ile Düşünme Döngüsü:

```
[Problem Tanımı]
      |
      ▼
[Sözde Kod (Pseudocode) Taslağı] ──> "Adım 1: Girdiyi doğrula, Adım 2: Set kullanarak tekilleştir"
      |
      ▼
[AI İle Mantık & Karmaşıklık Denetimi] ──> "Burada O(N^2) var, hash map kullanalım mı?"
      |
      ▼
[Temiz Kod Üretimi & Testler]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Algoritma Uzmanı
Bir kullanıcı listesindeki mükerrer (duplicate) e-posta adreslerini temizleyip en son kayıt olanı saklayan bir sistem tasarlayacağız.

Lütfen hemen Python veya JavaScript kodu YAZMA.
Bunun yerine:
1. Problemin adım adım SÖZDE KODUNU (pseudocode) çıkar.
2. Zaman karmaşıklığını (Time Complexity) hesapla.
3. Bellek kullanımını minimize etmek için alternatif bir yaklaşım var mı tartış.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü: İç içe döngülerle O(n^2) çalışan kontrolsüz arama
def deduplicate_users(users):
    unique = []
    for u in users:
        # Her eleman için listeyi baştan sona tarar: 100,000 kullanıcıda sistem
kilitlenir!
        found = False
        for ex in unique:
            if ex['email'] == u['email']:
                found = True
                break
        if not found:
            unique.append(u)
    return unique
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Sözde Koddan Üretilmiş O(n) Performanslı Versiyon
from typing import List, Dict, Any

def deduplicate_users_latest(users: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
    Kullanıcıları e-posta adresine göre tekilleştirir.
    En güncel 'created_at' damgasına sahip olan kaydı O(N) zaman karmaşıklığında
saklar.
    """
    latest_by_email: Dict[str, Dict[str, Any]] = {}

    for user in users:
        email = user.get("email", "").strip().lower()
        if not email:
            continue  # Geçersiz e-postaları atla

        existing = latest_by_email.get(email)
        if existing is None or user.get("created_at", 0) >
existing.get("created_at", 0):
            latest_by_email[email] = user

    return list(latest_by_email.values())
```

💡 **Kod Analizi:** Sözde kod aşamasında hash map (sözlük) kullanmaya karar verildiği için arama süresi O(N^2)'den O(N)'e düşürülmüştür. Ayrıca e-posta normalizasyonu (lowercase) ve boş değer koruması eklenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** AI'a 'bunu çöz' deyip çıkan karmaşık kodu anlamadan kabul etmek.
  - **Çözüm:** Her zaman önce algoritmanın Türkçe/İngilizce adımlarını yazdırın.
- **Tuzak:** Büyük veri setlerinde nested loop (iç içe döngü) tuzaklarına düşmek.
  - **Çözüm:** Veri yapısı olarak liste yerine Hash Map (Dict/Set) tercih edin.
- **Tuzak:** E-posta veya kullanıcı adı karşılaştırmalarında 'case-sensitivity'yi unutmak.
  - **Çözüm:** String verileri her zaman `.strip().lower()` ile normalize edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 7: Sözde Kod ile Düşünmek - AI ile Algoritma Tasarımı*

- 🖥️ **Slayt 1:** Neden doğrudan kod yazmak en büyük junior hatasıdır?
- 🖥️ **Slayt 2:** Sözde kod (Pseudocode) nedir ve nasıl yapılandırılır?
- 🖥️ **Slayt 3:** O(N^2) felaketinden O(N) zarafetine geçiş.
- 🖥️ **Slayt 4:** Canlı demo: E-ticaret sepet hesaplama algoritmasını sözde kodla tasarlamak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Terminal veya chat ekranında bir LeetCode orta seviye problem verin. 'Kod yazma, sadece 4 maddelik sözde kod çıkar' deyin. Çıkan sözde kodu adım adım onaylayıp ardından tek seferde temiz fonksiyonu ürettirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bir sonraki geliştireceğiniz fonksiyonda tek satır kod yazmadan önce defterinize veya not uygulamanıza 5 maddelik sözde kod yazın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Algoritma tasarımında sözde kod (pseudocode) kullanmanın en kritik faydası nedir? - A) Kodun derlenme süresini sıfıra indirmesi - B) Dil söz dizimi karmaşasına girmeden saf iş mantığını ve sınır durumlarını doğrulamaya imkan tanıması - C) Sadece Python dilinde çalışması - D) Veritabanı lisans maliyetlerini düşürmesi > **Doğru Yanıt:** B > **Açıklama:** Sözde kod, geliştiricinin ve yapay zekanın söz dizimi (syntax) yerine mantık akışına ve karmaşıklığa odaklanmasını sağlar.

Soru 2: 100.000 kullanıcılı bir listede tekilleştirme yaparken iç içe iki döngü kullanmanın tehlikesi nedir? - A) Hiçbir tehlikesi yoktur, en modern yöntemdir - B) O(N^2) zaman karmaşıklığı sebebiyle 10 milyar işlem gerektirip sunucuyu kilitlemesi - C) Sadece CSS hatalarına yol açması - D) Değişken isimlerinin

silinmesi > **Doğru Yanıt:** B > **Açıklama:** 100.000 eleman için N^2 işlem 10.000.000.000 adıma tekabül eder ve işlemciyi dakikalarca dondurur. O(N) hash map yaklaşımı ise milisaniyeler sürer.